Adaptive Beamsteering for Target Tracking Using a Phased Array Receiver: Simulation and Performance Analysis

Josiah Germain

***Abstract*—This work investigates adaptive beamsteering for target tracking using a uniform linear phased array receiver through a comprehensive MATLAB-based simulation framework. The array is modeled as a dynamic receiver that continuously steers its beam to follow a moving target, while incorporating practical system limitations including measurement noise, system lag, and sidelobe interference.**

Tracking performance is evaluated using normalized angular error and root-mean-square (RMS) error relative to the array half-power beamwidth (HPBW). In addition to baseline tracking behavior, parametric studies are conducted to examine the impact of beamwidth, system lag, noise level, and element spacing on tracking accuracy. Results demonstrate that narrower beamwidths significantly improve tracking performance due to increased angular resolution; however, system lag introduces dominant systematic errors that limit achievable accuracy. Furthermore, increased element spacing beyond $\lambda/2$ degrades performance due to elevated sidelobe levels and the onset of grating lobes.

The simulation highlights key trade-offs between spatial resolution, temporal response, and interference robustness in phased array tracking systems. These results provide insight into practical design considerations and motivate the use of advanced adaptive beamforming and filtering techniques for improved tracking performance.

***Index Terms*—Phased arrays, beamsteering, target tracking, antenna arrays, beamwidth, sidelobe level, MATLAB simulation.**

I. Introduction

Phased array antennas enable electronic control of beam direction through phase manipulation across multiple radiating elements. Unlike mechanically steered antennas, phased arrays provide rapid beam steering, making them well-suited for applications such as radar and wireless communication systems [2], [7].

In receiver-based tracking systems, the antenna must continuously estimate the direction of arrival (DoA) of an incoming signal and adjust its beam accordingly. While ideal beamsteering assumes perfect knowledge and instantaneous response, practical systems are affected by limitations such as noise, processing delay, and interference from sidelobes [2], [9].

This work develops a simulation framework to analyze how these non-ideal effects impact tracking performance. In particular, the relationship between array parameters—such as beamwidth and sidelobe level—and tracking accuracy is investigated. The goal is to connect fundamental antenna theory with system-level tracking behavior.

II. Theory

A. Array Factor and Beam Steering

For a uniform linear array (ULA) of N elements with spacing d, the array factor is given by [1]

$$AF(\theta) = \sum_{n=0}^{N-1} e^{jn(kd\sin\theta+\beta)} \tag{1}$$

where $k = \frac{2\pi}{\lambda}$ is the wavenumber and β is the progressive phase shift used for beam steering.

The beam is steered by selecting β such that

$$\beta = -kd\sin(\theta_0) \tag{2}$$

where θ_0 is the desired steering angle.

B. Beamwidth and Resolution

The half-power beamwidth (HPBW) defines the angular width of the main lobe and is inversely proportional to the array aperture [1]:

$$\text{HPBW} \propto \frac{\lambda}{Nd} \tag{3}$$

A narrower beamwidth corresponds to higher angular resolution, allowing more precise localization of a target.

C. Sidelobe Level and Interference

Sidelobes represent radiation in undesired directions and can introduce interference from off-axis signals. The sidelobe level (SLL) is defined as the ratio of the largest sidelobe to the main lobe peak [1]. In practical systems, sidelobes can contribute to measurement uncertainty and degraded tracking performance [2].

D. Tracking Error Model

Tracking error is defined as

$$e(t) = \theta_{\text{steer}}(t) - \theta_{\text{true}}(t) \tag{4}$$

To account for resolution limits, the error is normalized by HPBW:

$$e_{\text{norm}}(t) = \frac{e(t)}{\text{HPBW}} \tag{5}$$

Overall performance is quantified using RMS error [8]:

$$e_{\text{RMS}} = \sqrt{\frac{1}{T}\sum e_{\text{norm}}^2(t)} \tag{6}$$

III. Simulation Methodology

A MATLAB simulation was developed to model a phased array tracking a moving target. The target motion was defined as a sinusoidal function:

$$\theta_{\text{target}}(t) = 20\sin(2\pi \cdot 0.5t) \tag{7}$$

The following real-world effects were incorporated:

- **Measurement Noise:** Modeled as Gaussian noise [9]
- **System Lag:** Delay in beam steering due to processing and hardware limitations [2]
- **Sidelobe Interference:** Modeled as additional noise proportional to SLL

This model assumes ideal element behavior and neglects mutual coupling effects, which can further distort the radiation pattern in practical arrays [3].

IV. Results and Discussion

A. Time-Domain Tracking Behavior

The time-domain tracking performance of the phased array system is shown in Fig. 1. The steered beam closely follows the sinusoidal target trajectory, demonstrating the effectiveness of electronic beamsteering. However, a noticeable phase offset is present between the target and the steered beam, which is primarily caused by system lag.

In addition to lag-induced delay, high-frequency fluctuations are observed in the steered response. These variations arise from measurement noise and sidelobe-induced interference, which corrupt the estimated direction of arrival. This behavior is consistent with practical array systems, where noise and interference limit tracking fidelity [2], [9].

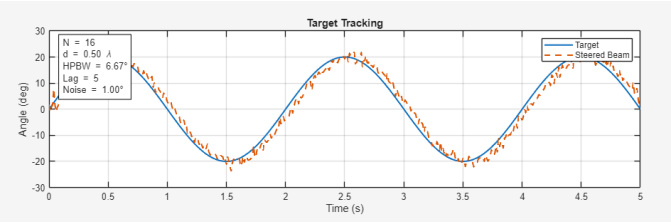


Fig. 1. Target and steered beam angles versus time ($N = 16$, HPBW = 6.67°, lag = 5 samples, noise $\sigma = 1°$).

B. Tracking Error Characteristics

The corresponding normalized tracking error is shown in Fig. 2. The error signal exhibits both structured and random components. The low-frequency oscillation is directly attributable to lag, which introduces a delayed response to changes in the target position. This aligns with classical tracking system behavior, where latency produces systematic estimation error [8].

Superimposed on this trend is a stochastic component caused by measurement noise and sidelobe interference. The RMS tracking error of approximately 0.3845 indicates that the system error is on the order of 38% of the beamwidth, which is significant but expected under non-ideal conditions.



Fig. 2. Normalized tracking error over time (RMS = 0.3845).

C. Effect of Beamwidth on Tracking Accuracy

To quantify the relationship between array resolution and tracking performance, RMS error was evaluated as a function of HPBW for multiple noise levels. The results are shown in Fig. 3.

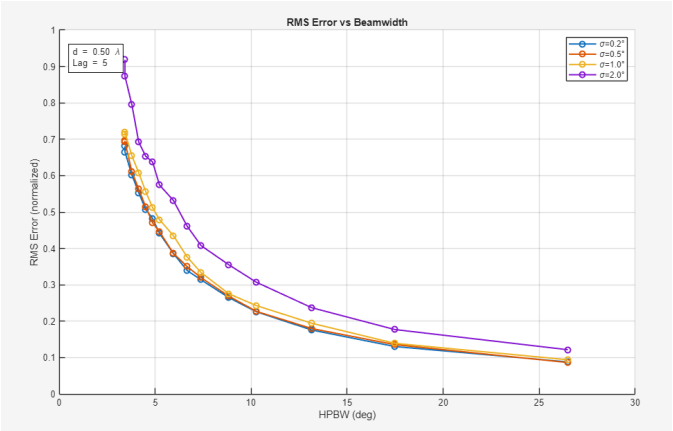


Fig. 3. RMS tracking error versus HPBW for multiple noise levels.

A clear inverse relationship is observed: as beamwidth decreases, RMS error decreases. This result directly confirms theoretical expectations, since narrower beams provide higher angular resolution [1].

Additionally, increasing noise levels shift the curves upward, indicating degraded tracking performance. However, the relative improvement from reducing beamwidth remains consistent across all noise levels, suggesting that array aperture is a dominant factor in tracking accuracy.

D. Effect of System Lag

To further investigate system dynamics, RMS error was evaluated as a function of beamwidth for multiple lag values. The results are shown in Fig. 4.

The results show that increasing lag significantly degrades tracking performance. For small lag values, the system is able to closely follow the target, resulting in lower RMS error. However, as lag increases, the system response becomes increasingly delayed, leading to larger tracking errors.

Interestingly, the benefit of narrower beamwidth is reduced at higher lag values. This occurs because system latency dominates the error, limiting the advantage gained from improved spatial resolution. This highlights an important system-level trade-off: improving antenna resolution alone is insufficient if temporal response is not also optimized [4].

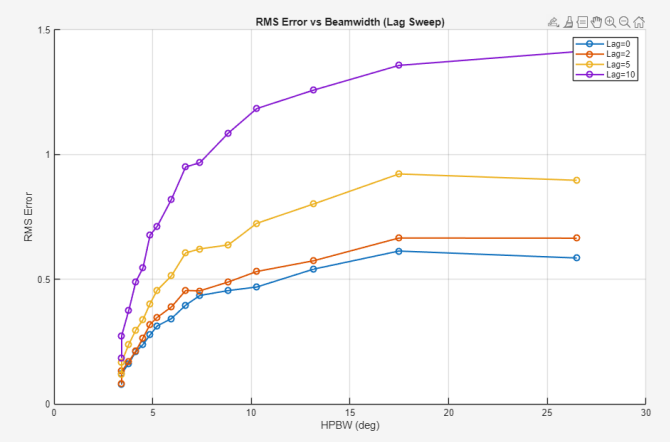


Fig. 4. RMS tracking error versus HPBW for different lag values.

E. Effect of Element Spacing

The impact of element spacing on tracking performance is shown in Fig. 5, where RMS error is plotted for $d = \lambda/2$ and $d = 0.75\lambda$.

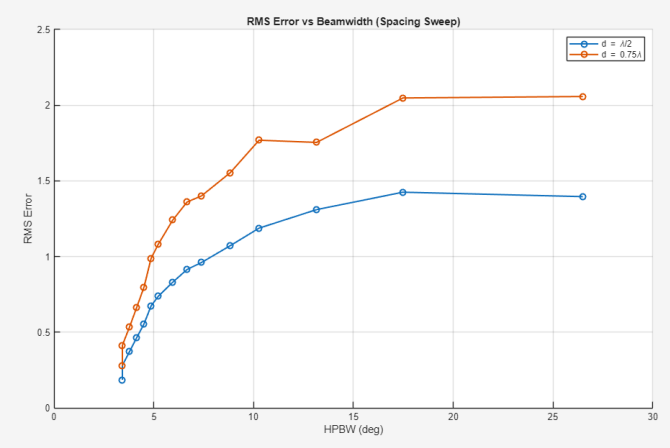


Fig. 5. RMS tracking error versus HPBW for different element spacings.

The results indicate that increasing element spacing leads to higher RMS error across all beamwidth values. While larger spacing can reduce beamwidth for a fixed number of elements, it also introduces grating lobes when $d > \lambda/2$, which significantly increase sidelobe levels [1].

These elevated sidelobes act as additional sources of interference, degrading the accuracy of angle estimation. This behavior is consistent with classical array theory, where excessive spacing results in spatial aliasing and reduced directional selectivity.

F. Comparison with Advanced Methods

The results presented in this work are based on conventional beamsteering using fixed phase shifts. More advanced adaptive beamforming techniques, such as the Capon method and Frost algorithm, offer improved interference suppression and higher resolution [5], [6].

These methods dynamically adjust array weights to minimize output power or enforce constraints, allowing them to suppress sidelobes and noise more effectively than fixed beamforming. Similarly, adaptive filtering approaches [10] and optimal array processing techniques [4] provide further improvements in tracking performance.

G. Overall Design Trade-Offs

The combined results highlight several key trade-offs in phased array tracking systems:

- Narrower beamwidth improves tracking accuracy but increases system complexity
- Measurement noise degrades performance across all configurations
- System lag introduces dominant low-frequency tracking error
- Increased element spacing can introduce grating lobes and degrade performance
- Sidelobes act as additional noise sources, reducing estimation accuracy

These trade-offs emphasize the need for a balanced design that considers both spatial resolution and temporal responsiveness.

V. Conclusion

This work presented a simulation-based study of adaptive beamsteering for target tracking using a uniform linear phased array receiver. By incorporating practical system limitations—including measurement noise, system lag, and sidelobe interference—the analysis provides a more realistic assessment of tracking performance beyond idealized beamsteering models.

The results demonstrate that beamwidth is a primary factor governing tracking accuracy. As shown in the beamwidth sweep, narrower beams consistently produce lower RMS tracking error due to improved angular resolution. This aligns with classical antenna theory, where increased aperture size enhances directivity and localization capability [1].

However, the study also shows that spatial resolution alone does not fully determine system performance. The lag sweep results reveal that system delay introduces a dominant source of error, producing systematic tracking offsets that cannot be mitigated by reducing beamwidth alone. At higher lag values, the benefit of narrower beams diminishes, indicating that temporal response becomes the limiting factor in tracking accuracy. This highlights the importance of considering both spatial and temporal system design, as emphasized in optimal array processing theory [4].

The spacing sweep further demonstrates that increasing element spacing beyond $\lambda/2$ degrades tracking performance despite its effect on beamwidth. This is due to the emergence of grating lobes and elevated sidelobe levels, which introduce additional interference and ambiguity in angle estimation. These findings reinforce the importance of maintaining appropriate element spacing to avoid spatial aliasing and preserve directional selectivity [1], [2].

Overall, the results highlight key trade-offs in phased array tracking systems: improving beamwidth enhances accuracy, but noise, sidelobes, and lag introduce both random and systematic errors that limit performance. Among these factors, system lag and sidelobe interference play a critical role in practical implementations, often constraining achievable accuracy more than resolution alone.

Future work may extend this study by incorporating adaptive beamforming techniques such as the Capon method or Frost algorithm, which can suppress interference and improve resolution. Additionally, integrating more advanced tracking algorithms, such as Kalman filtering, could further reduce the impact of noise and lag, leading to more robust and accurate tracking performance in dynamic environments [8], [10].

APPENDIX

```
% Phased Array Antenna Simulation (Uniform Linear
    Array)
%
    ------------------------------------------------------

% Features:
%   - Array factor plotting
%   - Beamwidth + SLL metrics
%   - Target tracking simulation
%   - RMS tracking error
%   - Multi-curve RMS vs HPBW (noise + lag sweeps)
%   - Optional spacing sweep (grating lobes insight)
%
% Author: Josiah Germain

% Housekeeping
close all; clear; clc;

% ==========================
% USER SETTINGS
% ==========================
runTracking = true;
useNormalization = true;
useDecibelPlot  = false;

% Array parameters
lambda = 0.3;
K = 2*pi/lambda;
d = lambda/2;
N = 16;

% ==========================
% TRACKING MODE
% ==========================
if runTracking

    % Time + motion
    t_total = 5;
    dt = 0.01;
    t = 0:dt:t_total;
    theta_target = 20 * sin(2*pi*0.5*t);

    % Imperfections (baseline case)
    lag_steps = 5;
    noise_std = 1.0;

    % Reference beam
    theta_0 = 0;
    B = -K*d*sin(theta_0);

    % Get metrics
    [~, ~, HPBW, ~, SLL_dB, ~, ~] = ...
        computeArrayFactor(K, d, N, B, lambda, true)
            ;

    SLL_linear = 10^(SLL_dB/20);

    % Run tracking
    [theta_steered, tracking_error, RMS_error] = ...
        runTrackingSimulation(theta_target, HPBW,
            SLL_linear, lag_steps, noise_std);

    % ==========================
    % PLOTS
    % ==========================
    figure;

    subplot(2,1,1)
    plot(t, theta_target, 'LineWidth', 1.5); hold on
        ;
    plot(t, theta_steered, '--', 'LineWidth', 1.5);
    xlabel('Time (s)');
    ylabel('Angle (deg)');
    title('Target Tracking');
    legend('Target','Steered Beam');
    grid on;

    text(0.02,0.95,{
        sprintf('N = %d', N)
        sprintf('d = %.2f \\lambda', d/lambda)
        sprintf('HPBW = %.2 f  ', HPBW)
        sprintf('Lag = %d', lag_steps)
        sprintf('Noise = %.2 f  ', noise_std)
    },'Units','normalized','VerticalAlignment','top
        ',...
      'BackgroundColor','w','EdgeColor','k');

    subplot(2,1,2)
    plot(t, tracking_error, 'LineWidth', 1.5);
    xlabel('Time (s)');
    ylabel('Normalized Error');
    title('Tracking Error');
    grid on;

    text(0.02,0.95,{
        sprintf('RMS = %.4f', RMS_error)
        sprintf('HPBW = %.2 f  ', HPBW)
        sprintf('SLL = %.2f dB', SLL_dB)
    },'Units','normalized','VerticalAlignment','top
        ',...
      'BackgroundColor','w','EdgeColor','k');

    % ==========================
    % NOISE SWEEP
    % ==========================
    N_values = 4:2:32;
    noise_levels = [0.2 0.5 1.0 2.0];

    HPBW_values = zeros(size(N_values));
    RMS_matrix = zeros(length(noise_levels), length(
        N_values));

    for n_idx = 1:length(noise_levels)

        noise_std = noise_levels(n_idx);

        for i = 1:length(N_values)

            N_temp = N_values(i);

            [~, ~, HPBW, ~, SLL_dB, ~, ~] = ...
                computeArrayFactor(K, d, N_temp, B,
                    lambda, true);

            SLL_linear = 10^(SLL_dB/20);

            [~, ~, RMS_error] = ...
                runTrackingSimulation(theta_target,
                    HPBW, SLL_linear, lag_steps,
                    noise_std);

            HPBW_values(i) = HPBW;
```

```
        RMS_matrix(n_idx,i) = RMS_error;
    end
end

[HPBW_values, idx] = sort(HPBW_values);
RMS_matrix = RMS_matrix(:, idx);

figure; hold on; grid on;
for i = 1:length(noise_levels)
    plot(HPBW_values, RMS_matrix(i,:), 'o-', '
        LineWidth', 1.5);
end

xlabel('HPBW (deg)');
ylabel('RMS Error');
title('RMS Error vs Beamwidth (Noise Sweep)');
legend(arrayfun(@(x) sprintf('\\sigma=%.1f ',x)
    , noise_levels,'UniformOutput',false));

% ==========================
% LAG SWEEP
% ==========================
lag_values = [0 2 5 10];
RMS_lag = zeros(length(lag_values), length(
    N_values));

for l_idx = 1:length(lag_values)

    lag_steps = lag_values(l_idx);

    for i = 1:length(N_values)

        N_temp = N_values(i);

        [~, ~, HPBW, ~, SLL_dB, ~, ~] = ...
            computeArrayFactor(K, d, N_temp, B,
                lambda, true);

        SLL_linear = 10^(SLL_dB/20);

        [~, ~, RMS_error] = ...
            runTrackingSimulation(theta_target,
                HPBW, SLL_linear, lag_steps,
                noise_std);

        RMS_lag(l_idx,i) = RMS_error;
    end
end

figure; hold on; grid on;
for i = 1:length(lag_values)
    plot(HPBW_values, RMS_lag(i,:), 'o-', '
        LineWidth', 1.5);
end

xlabel('HPBW (deg)');
ylabel('RMS Error');
title('RMS Error vs Beamwidth (Lag Sweep)');
legend(arrayfun(@(x) sprintf('Lag=%d',x),
    lag_values,'UniformOutput',false));

% ==========================
% ELEMENT SPACING SWEEP
% ==========================
d_values = [lambda/2, 0.75*lambda];
RMS_spacing = zeros(length(d_values), length(
    N_values));

for d_idx = 1:length(d_values)

    d_temp = d_values(d_idx);

    for i = 1:length(N_values)

        N_temp = N_values(i);

        [~, ~, HPBW, ~, SLL_dB, ~, ~] = ...
            computeArrayFactor(K, d_temp, N_temp
                , B, lambda, true);

        SLL_linear = 10^(SLL_dB/20);

        [~, ~, RMS_error] = ...
            runTrackingSimulation(theta_target,
                HPBW, SLL_linear, lag_steps,
                noise_std);

        RMS_spacing(d_idx,i) = RMS_error;
    end
end

figure; hold on; grid on;
for i = 1:length(d_values)
    plot(HPBW_values, RMS_spacing(i,:), 'o-', '
        LineWidth', 1.5);
end

xlabel('HPBW (deg)');
ylabel('RMS Error');
title('RMS Error vs Beamwidth (Spacing Sweep)');
legend({'d = \lambda/2','d = 0.75\lambda'});

end

% ==========================
% ARRAY FACTOR MODE
% ==========================
if ~runTracking

    theta_deg_input = 0;
    theta_0 = deg2rad(theta_deg_input);
    B = -K*d*sin(theta_0);

    [theta_deg, AF_plot, HPBW, FNBW, SLL_dB, idxL,
        idxR] = ...
        computeArrayFactor(K, d, N, B, lambda,
            useNormalization);

    figure; hold on; grid on;

    if useDecibelPlot
        plot(theta_deg, 20*log10(AF_plot),'LineWidth
            ',1.5);
        ylabel('dB');
    else
        plot(theta_deg, AF_plot,'LineWidth',1.5);
        ylabel('Magnitude');
    end

    xlabel('Angle (deg)');
    title('Array Factor');

    xline(theta_deg(idxL),'--');
    xline(theta_deg(idxR),'--');

end

% ==========================
% FUNCTION: TRACKING
% ==========================
function [theta_steered, tracking_error, RMS_error]
    = ...
    runTrackingSimulation(theta_target, HPBW,
        SLL_linear, lag_steps, noise_std)

    theta_steered = zeros(size(theta_target));
    tracking_error = zeros(size(theta_target));
    hist = zeros(size(theta_target));

    for k = 1:length(theta_target)

        theta_true = theta_target(k);

        noise = noise_std*randn;
```

```
        interference = SLL_linear*randn;

        meas = theta_true + noise + interference;
        hist(k) = meas;

        if k > lag_steps
            theta_steer = hist(k-lag_steps);
        else
            theta_steer = meas;
        end

        theta_steered(k) = theta_steer;

        tracking_error(k) = (theta_steer -
            theta_true)/HPBW;

    end

    RMS_error = sqrt(mean(tracking_error.^2));
end

% ==========================
% FUNCTION: ARRAY FACTOR
% ==========================
function [theta_deg, AF_plot, HPBW, FNBW, SLL_dB,
    idxL, idxR] = ...
    computeArrayFactor(K,d,N,B,lambda,
        useNormalization)

    theta = linspace(-pi/2, pi/2, 1000);

    AF = zeros(size(theta));
    for n=0:N-1
        AF = AF + exp(1i*n*(K*d*sin(theta)+B));
    end

    AF_mag = abs(AF);
    peak = max(AF_mag);

    [~, idxMax] = max(AF_mag);

    idxL = idxMax;
    while idxL>1 && AF_mag(idxL)>=0.707*peak
        idxL=idxL-1;
    end

    idxR = idxMax;
    while idxR<length(AF_mag) && AF_mag(idxR)
        >=0.707*peak
        idxR=idxR+1;
    end

    HPBW = rad2deg(theta(idxR)-theta(idxL));
    FNBW = 2*asind(lambda/(N*d));

    side = AF_mag;
    side(idxL:idxR)=0;
    SLL = max(side)/peak;
    SLL_dB = 20*log10(SLL);

    if useNormalization
        AF_plot = AF_mag/peak;
    else
        AF_plot = AF_mag;
    end

    theta_deg = rad2deg(theta);
end
```

REFERENCES

[1] C. A. Balanis, *Antenna Theory: Analysis and Design*, 4th ed. Hoboken, NJ, USA: Wiley, 2016.

[2] R. J. Mailloux, *Phased Array Antenna Handbook*, 3rd ed. Norwood, MA, USA: Artech House, 2018.

[3] D. M. Pozar, "The active element pattern," *IEEE Trans. Antennas Propag.*, vol. 42, no. 8, pp. 1176–1178, Aug. 1994.

[4] H. L. Van Trees, *Optimum Array Processing*. New York, NY, USA: Wiley, 2002.

[5] J. Capon, "High-resolution frequency-wavenumber spectrum analysis," *Proc. IEEE*, vol. 57, no. 8, pp. 1408–1418, Aug. 1969.

[6] O. L. Frost III, "An algorithm for linearly constrained adaptive array processing," *Proc. IEEE*, vol. 60, no. 8, pp. 926–935, Aug. 1972.

[7] M. I. Skolnik, *Introduction to Radar Systems*, 3rd ed. New York, NY, USA: McGraw-Hill, 2001.

[8] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. New York, NY, USA: Wiley, 2001.

[9] T. S. Rappaport, *Wireless Communications: Principles and Practice*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2002.

[10] S. Haykin, *Adaptive Filter Theory*, 5th ed. Upper Saddle River, NJ, USA: Pearson, 2013.